



LI.FI

LI.FI Security Review

LiFiIntentEscrowFacetV2.sol(v1.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

June 30, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Low Risk	4
6.1.1	Non-EVM receiver combined with a destination call produces an un-fillable callback	4
6.2	Informational	4
6.2.1	Docs omit that the same-network guard is intentionally absent	4
6.2.2	V2's <code>LIFI_INTENT_ESCROW_SETTLER()</code> getter is shadowed by V1's identical selector	4
6.2.3	Swap-path output scaling is linear and undocumented	5
6.2.4	Make <code>MULTIPLIER_BASE</code> non-public to drop a getter selector and its collision handling	5
6.2.5	Validate <code>outputAmountMultiplier != 0</code> before depositing/swapping	5
6.2.6	"Receiver is validated on-chain" is overstated for the destination-call path	6
6.2.7	Source-swap output token is not required to match <code>bridgeData.sendingAssetId</code>	6

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/LiFiIntentEscrowFacetV2.sol(v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 8 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 8 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	11aad267
Audit Timeline	June 28, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Gas Optimizations	0
Informational	7
Total Issues	8

6 Findings

6.1 Low Risk

6.1.1 Non-EVM receiver combined with a destination call produces an un-fillable callback

Context: [LiFiIntentEscrowFacetV2.sol#L191](#)

Description: V2 permits `bridgeData.receiver == NON_EVM_ADDRESS` and `dstCallSwapData.length > 0` simultaneously. In that case the non-EVM branch leaves `_lifiIntentData.recipient` unvalidated except for non-zero, and the destination-call branch ABI-encodes that full bytes32 recipient into the callback data.

On the destination chain, `ReceiverOIF` decodes that slot as an address (`src/Periphery/ReceiverOIF.sol:81`), and Solidity's ABI decoder reverts whenever the value's upper 12 bytes are non-zero, which is the normal case for a true 32-byte non-EVM address.

The result is a callback that can never be filled (the order's input is escrowed until expiry and then refunded). The combination is meaningless to begin with, since `ReceiverOIF` is an EVM contract that cannot execute on a non-EVM destination, so it only arises from malformed calldata.

Recommendation: Reject the combination on-chain: revert when `_bridgeData.receiver == NON_EVM_ADDRESS` and `dstCallSwapData.length > 0` or, if you prefer to keep the facet permissive, document explicitly that destination calls are EVM-destination-only.

LI.FI: Fixed in [186374d](#)

Researcher: Verified fix.

6.2 Informational

6.2.1 Docs omit that the same-network guard is intentionally absent

Context: [LiFiIntentEscrowFacetV2.sol#L24](#), [LiFiIntentEscrowFacetV2.md](#)

Description: The "LiFi Data" boilerplate in `docs/LiFiIntentEscrowFacetV2.md:106` implies the standard `Validatable` checks apply, but V2 uses a custom modifier that only enforces `receiver != 0` and `minAmount != 0` and the `destinationChainId == block.chainid` (`CannotBridgeToSameNetwork`) guard is deliberately omitted, so same-chain intents are allowed (confirmed by the overridden `testBase_Revert_*SameChainId` tests).

Recommendation: Add a sentence to the docs (and ideally a note on the custom modifier's `NatSpec`) clarifying that validation is limited to non-zero receiver and `minAmount`, and that the same-network check is intentionally not applied because same-chain intents are supported.

LI.FI: Fixed in [5c19891](#)

Researcher: Verified fix.

6.2.2 V2's `LIFI_INTENT_ESCROW_SETTLER()` getter is shadowed by V1's identical selector

Context: [LiFiIntentEscrowFacetV2.sol#L47](#)

Description: V1 and V2 expose the same "`LIFI_INTENT_ESCROW_SETTLER()`" getter selector. V2's diamond cut excludes it (to avoid orphaning V1's entrypoints via a `Replace`), so the diamond keeps resolving the getter to V1 and returning V1's settler, not V2's.

Harmless today since both are built from the same `.lifiEscrowInputSettler` config and no on-chain path uses the getter; it only misleads off-chain readers, and only if V1 and V2 are ever given different settlers.

Recommendation: Document the shared-settler assumption in the update script, and add a deploy-time assertion that V2's constructor settler equals the settler V1 currently serves while the colliding selector stays excluded.

If the diamond must expose V2's own value, rename the V2 getter (e.g. `LIFI_INTENT_ESCROW_SETTLER_V2()`) so it has a distinct, registrable selector.

LI.FI: Fixed in [b165772](#)

Researcher: Verified fix.

6.2.3 Swap-path output scaling is linear and undocumented

Context: [LiFiIntentEscrowFacetV2.sol#L116](#), [LiFiIntentEscrowFacetV2.sol#L152](#)

Description: On the swap entrypoint, positive slippage is not refunded; the realized swap output funds the intent and the committed output is scaled off the backend's constant multiplier:

```
uint256 effectiveOutputAmount =  
    (swapOutcome * _lifiIntentData.outputAmountMultiplier) / MULTIPLIER_BASE;
```

This holds the intent's input $\rightarrow$ output exchange rate constant at $\text{outputAmountMultiplier} / 1e18$, regardless of the realized input size.

That assumption is exact for a purely proportional rate, but a solver's fillable rate is not strictly proportional: it also reflects flat/tiered components (gas, fixed fees) and liquidity-dependent pricing that don't scale linearly with size.

Recommendation: This is acceptable as designed. Make the assumption explicit so integrators understand the trade-off, by documenting in the outputAmount / function NatSpec.

LI.FI: Documented in [99490c5](#)

Researcher: Verified fix.

6.2.4 Make MULTIPLIER_BASE non-public to drop a getter selector and its collision handling

Context: [LiFiIntentEscrowFacetV2.sol#L51](#)

Description: MULTIPLIER_BASE is declared `uint256 public constant`. Because it is constant, every in-contract use is inlined at compile time, so visibility has zero runtime cost; the only effect of public is a synthesized getter whose selector is added to the facet.

That MULTIPLIER_BASE() selector collides with AcrossFacetV4/AcrossV4SwapFacet, which is exactly why it must be excluded from the diamond cut in `UpdateLiFiIntentEscrowFacetV2.s.sol:16`.

Recommendation: Change MULTIPLIER_BASE to internal (or private) and remove `excludes[0] = facet.MULTIPLIER_BASE.selector;` from `getExcludes()` (sizing the array to one entry).

LI.FI: Fixed in [b165772](#)

Researcher: Verified fix.

6.2.5 Validate outputAmountMultiplier != 0 before depositing/swapping

Context: [LiFiIntentEscrowFacetV2.sol#L116](#), [LiFiIntentEscrowFacetV2.sol#L149](#)

Description: Both entrypoints do their expensive work first and only reject a zero output later: the non-swap path calls `LibAsset.depositAsset` and the swap path runs the full `_depositAndSwap` before `_startBridge()` computes `effectiveOutputAmount` and reverts `InvalidAmount`.

When `outputAmountMultiplier == 0`, `effectiveOutputAmount` is necessarily 0, so the call always reverts but only after the ERC20 `transferFrom` (non-swap) or the entire swap sequence (swap path) has already executed and burned gas.

`outputAmountMultiplier` is a plain calldata input that can be checked up front, before any token movement. (Note this only fast-fails the explicit-zero case; the `effectiveOutputAmount == 0` check at :186 must stay, since a non-zero multiplier can still floor to zero for tiny amounts.)

Recommendation: Add `if (_lifiIntentData.outputAmountMultiplier == 0) revert InvalidAmount();` at the top of both entrypoints.

LI.FI: Acknowledged. As written in the finding, this is a fail-fast check and will not change outcomes. Since `_lifiIntentData.outputAmountMultiplier == 0` is an obvious mistake and caught anyway, this will not be implemented.

Researcher: Acknowledged.

6.2.6 "Receiver is validated on-chain" is overstated for the destination-call path

Context: [LiFiIntentEscrowFacetV2.md](#)

Description: The claim in `LiFiIntentEscrowFacetV2.md` ("The receiver is still validated on-chain") holds only for the non-destination-call path.

When `dstCallSwapData.length != 0`, `_startBridge()` overwrites the OIF `MandateOutput.recipient` with `dstCallReceiver` and merely encodes the validated user recipient into callback data, with `dstCallReceiver` itself checked only for non-zero.

The funds are therefore delivered on-chain to `dstCallReceiver`, and the user recipient is honored only if that address is a genuine `ReceiverOIF` that decodes and forwards the callback. `ReceiverOIF` explicitly disclaims this responsibility and it "does not validate exterior calldata or execution," and bad swap data "WILL lead to a loss of funds ranging from complete loss to high slippage" (`src/Periphery/ReceiverOIF.sol:63`). So on the destination-call path the effective receiver is a backend-supplied address that is not bound on-chain to `bridgeData.receiver`, contradicting the unqualified documentation statement.

Recommendation: Narrow the docs statement to clarify that full on-chain receiver binding applies only when there is no destination call, and that on the destination-call path delivery depends on `dstCallReceiver` being a trusted `ReceiverOIF`.

LI.FI: Fixed in [674c7b0](#)

Researcher: Verified fix.

6.2.7 Source-swap output token is not required to match `bridgeData.sendingAssetId`

Context: [LiFiIntentEscrowFacetV2.sol#L132](#)

Description: On the swap entrypoint (`swapAndStartBridgeTokensViaLiFiIntentEscrowV2`), `_depositAndSwap()` measures the realized output of the final swap as the balance delta of `_swaps[last].receivingAssetId` and returns it as `swapOutcome`, which is written into `_bridgeData.minAmount`.

`_startBridge()` then approves the settler and registers the order input (`inputs[0]`) using a different field, `_bridgeData.sendingAssetId`, with that amount.

Nothing on-chain enforces that the swap's final receiving token actually equals `sendingAssetId`; their consistency is purely a backend convention.

Recommendation: Add a cheap explicit check in the swap entrypoint asserting `_swaps[_swaps.length - 1].receivingAssetId == _bridgeData.sendingAssetId`, making the deposited-token invariant explicit on-chain alongside the existing `minAmount` slippage floor.

LI.FI: This is generally not implemented for facets; The facet will revert. Acknowledged.

Researcher: Acknowledged.